

# Machine Learning, diferența dintre o abordare stocastică și una deterministă

Autori: Bonciu Vlad-Matei, Beșliu Stefano

Academia de Studii Economice București, Facultatea de Cibernetică, Statistică și Informatică Economică

# Repererele Principale

- *Istoria Învățării Automate*
- *Rețele Neurale*
- *K-Nearest-Neighbours (KNN)*
- *Comparația dintre cei doi algoritmi*

Noi am abordat obiectivul de recunoaștere ale unor cifre scrise de mână, o metoda clasică de demonstrație a capacităților de clasificare.

# Începuturile Învățării Automate

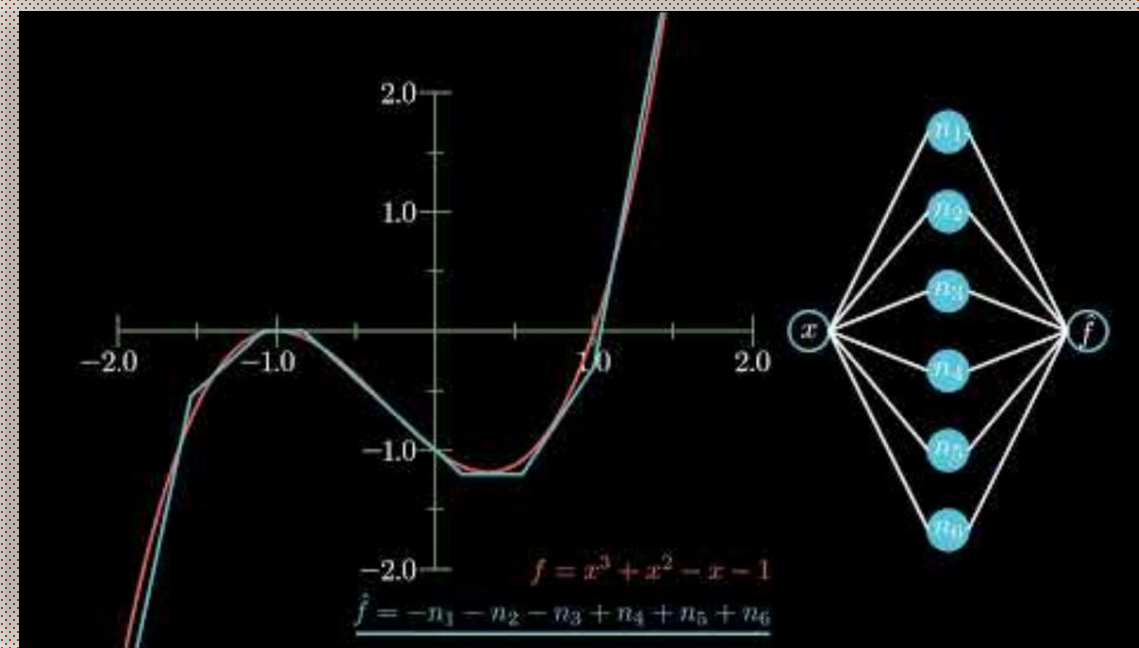
- *Frank Rosenblatt*, un remarcabil psiholog american, este considerat inițiatorul Deep Learning-ului și Rețelelor Neurale. Acesta, prin “Perceptronul”, a reușit să imite principiile biologice aferente învățării umane, într-un sistem de calcul. *Perceptronii săi, au fost inițial simulați pe un IBM 704, în 1957*
- Din anii 1980, dezvoltarea Machine Learning-ului încetinește, fiind limitat de capacitățile computaționale ale sistemelor de calcul.
- În 1990, însă, domeniul înflorește cu o serie de descoperiri revoluționare, care vor marca un nou început.
- Astăzi, Machine Learning-ul este declarat ca fiind „*Știința care face ca sistemele de calcul să acționeze, fără a fi programate în mod explicit*” (Stanford University).



*Convolutional Network Demo from 1989 – Yann LeCun – youtube.com*

# Rețele Neurale

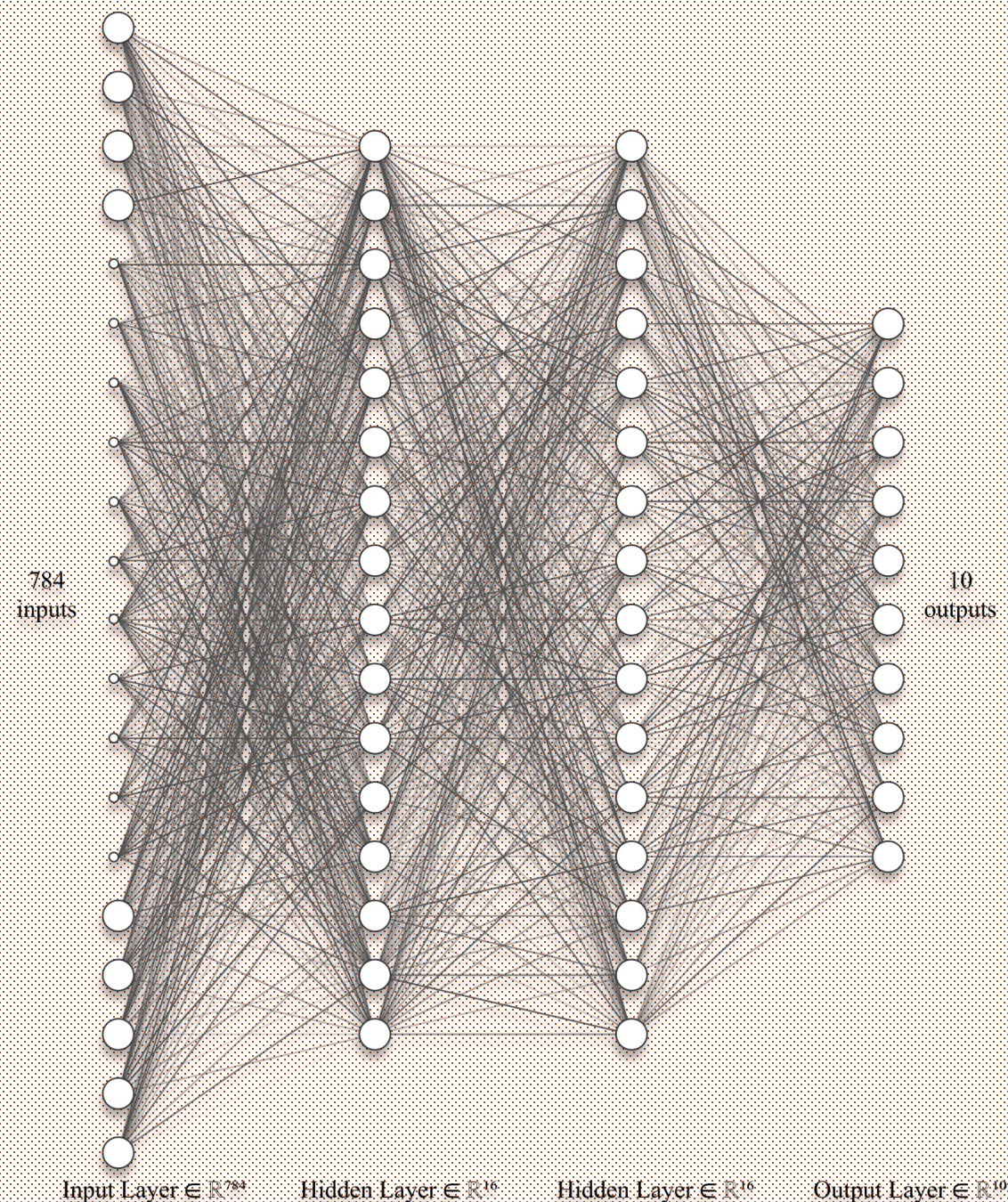
- Rețelele Neurale (*engl. Neural Networks, abreviate NN*), constituie punctul de plecare al tuturor algoritmilor și procedurilor moderne de învățare automată, fiind întâlnite în nenumărate tehnologii moderne, precum LLM-uri, CNN și Deep Fake
- *Au aplicabilitate aproape nemărginită* (sunt universal function approximators).
- Pot fi utilizate *atât pentru regresie cât și clasificare*
- Sunt compatibile atât cu Supervised Learning, cât și Unsupervised Learning
- La nivel intuitiv, *acestea pot fi văzute ca niște grafuri multi-partite*, unde fiecare mulțime e un strat al rețelei, menit să proceseze o caracteristică anume



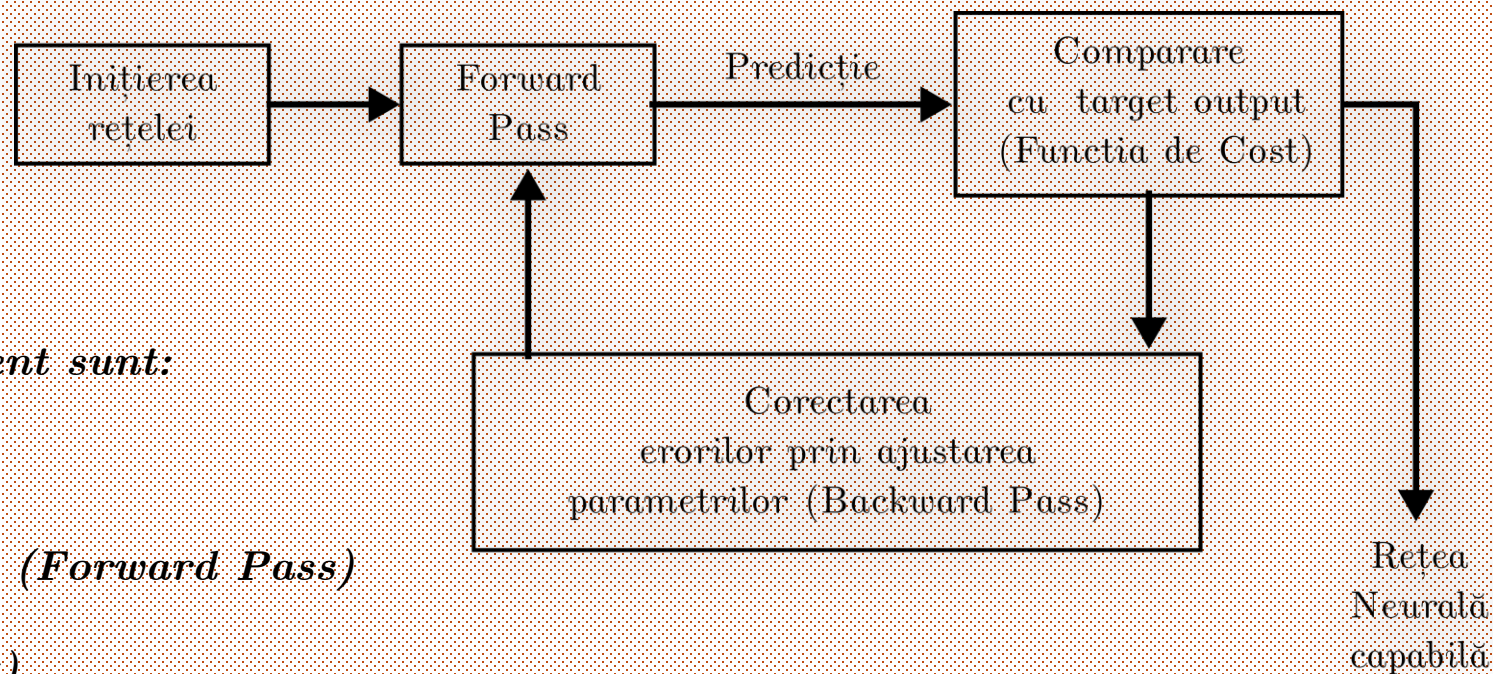
Visualization of the universal approximation theorem  
– Hugo Mougard – youtube.com

# Rețele Neurale

- Arhitectura pe care am ales să o dezvoltăm este cea de *Multi-Layer Perceptron (MLP)*, care formează rețele neurale de mai multe straturi, cu proprietatea de a procesa date și a le organiza strat cu strat.
- *La nivel matematic, aceștia pot fi considerați o funcție multivariabilă*, capabilă să filtreze datele primite, pentru a oferi un output relevant scopului rețelei.
- *Legăturile dintre straturi, sunt marcate cu niște ponderi, inițializate aleatoriu*, fapt ce face procesul de antrenare al acestui model unul stocastic.
- Astfel, în termeni simpli, o rețea neurală poate fi interpretată drept o funcție de mii sau miliarde de variabile, menită să evalueze un anumit context sau obiect



# Rețele Neurale



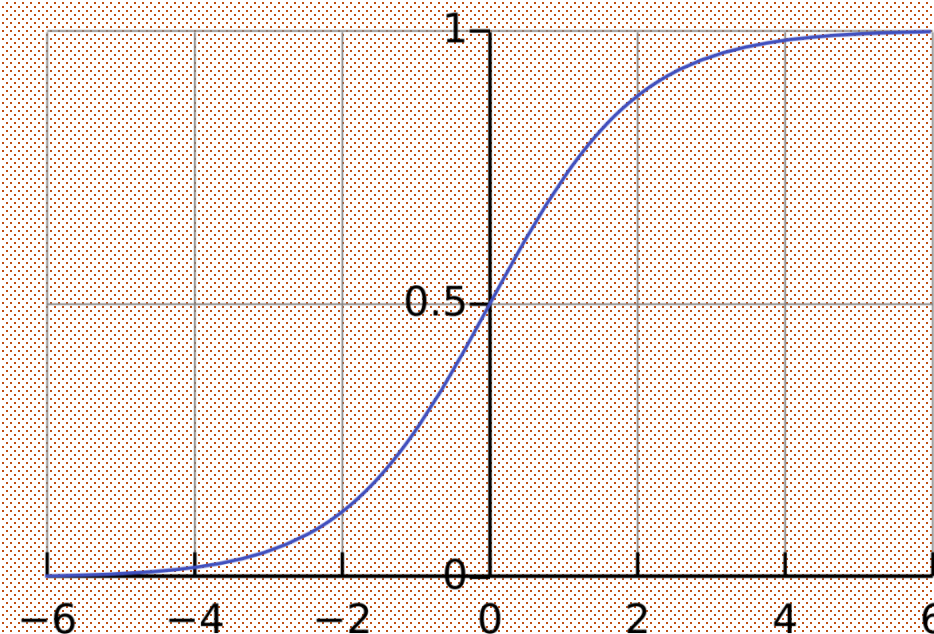
- *Pasii parcurși în etapa de antrenament sunt:*

- 1) Inițierea rețelei neurale cu valori aleatorii
- 2) Propagarea input-ului prin toate straturile (*Forward Pass*)
- 3) Evaluarea rezultatelor (*Funcția de Cost*)
  - Dacă suntem mulțumiți cu rezultatul → Finalizare program
  - Altfel trecem la pasul 4)
- 4) Ajustarea parametrilor pe baza rezultatului obținut (*Backpropagation*)
- 5) Trecem la pasul 2) pentru un alt input.

# Funcția de activare

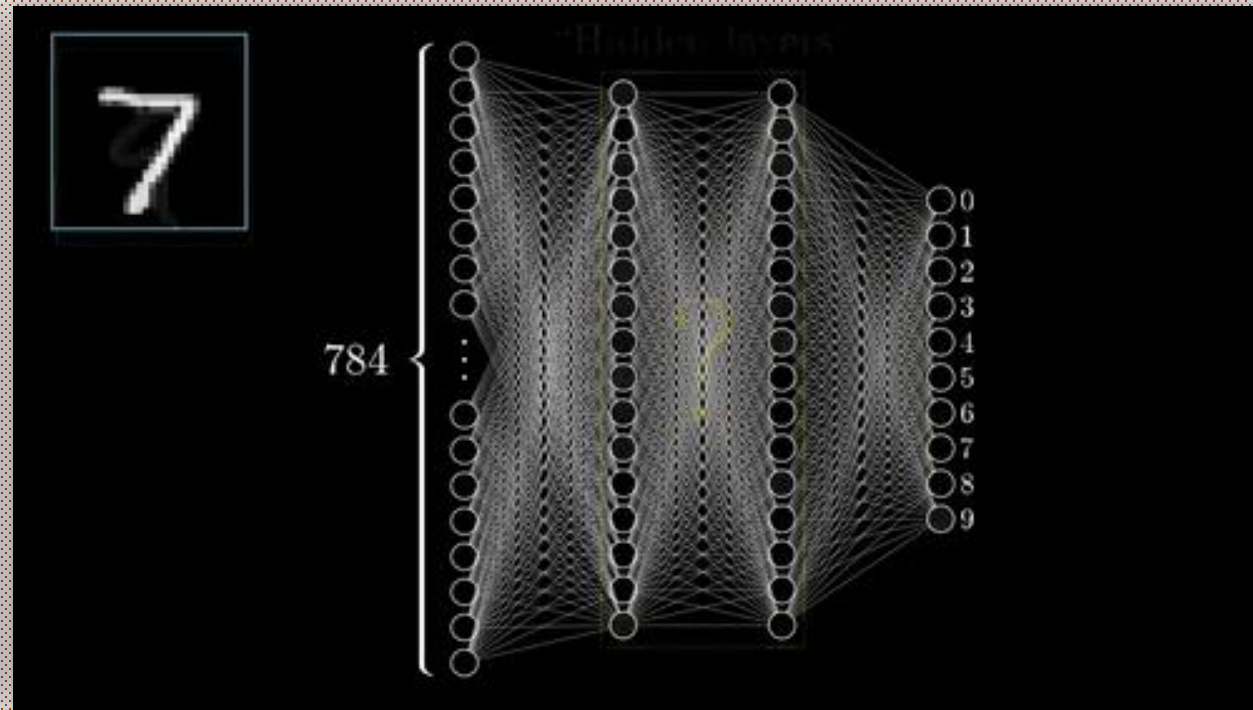
- *Funcția de activare* este operația pe care o aplicăm la sfârșitul operațiilor realizate de fiecare perceptron. Am decis să o utilizăm este cea **logistică** pentru proprietățile sale:
  - 1) **Imaginea [0;1]**, relevantă pentru determinarea gradului de activare al neuronilor
  - 2) **Ușor de implementat și computat** (atât ea cât și derivata sa)
- Astfel marcăm un neuron puternic activat cu o valoare apropiată de 1 și unul slab activat cu 0.
- Astfel, în termeni simpli, o rețea neurală poate fi interpretată drept o funcție de mii sau miliarde de variabile, menită să evalueze un anumit context sau obiect

$$\sigma(x) = \frac{1}{1+e^{-x}} = \frac{e^x}{1+e^x} = 1 - \sigma(-x)$$



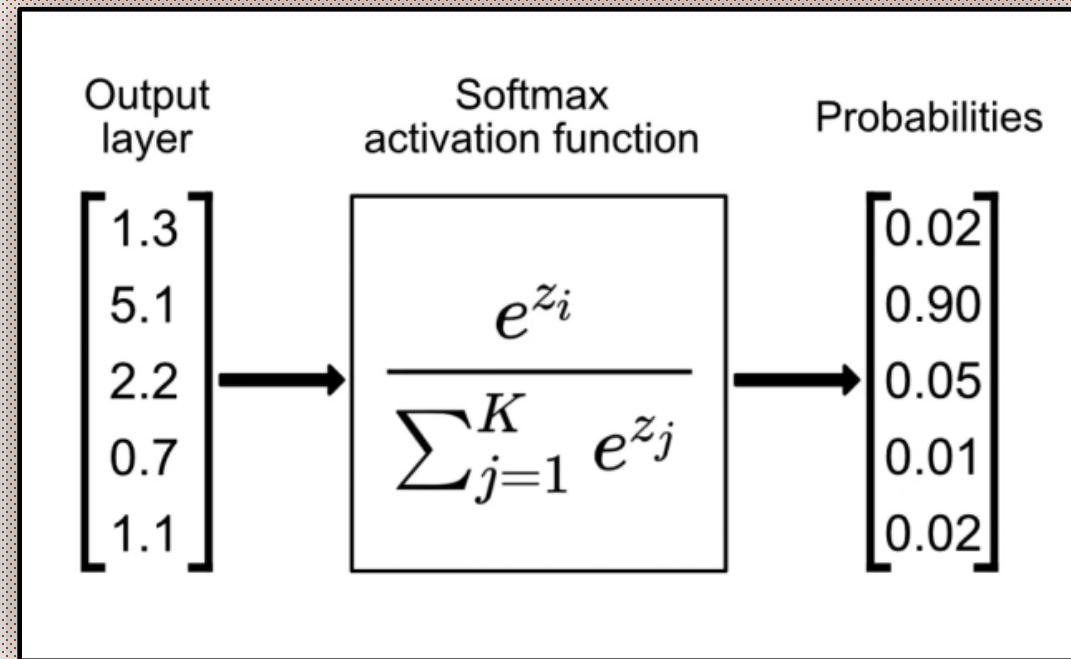
# Forward Pass

- Proprietatea NN-urilor de a procesa și transmite informații de la stratul de intrare, până la cel terminal.
- Importanța sa reiese din modul în care datele sunt percepute și alterate pe parcursul antrenamentelor
- Antrenamentul, este procesul de învățare al rețelei, unde în urma unui rezultat, aceasta își autoevaluează procesul



# Funcția Softmax

- *Pentru evaluarea calității rețelei, trebuie să transformăm rezultatele, într-o serie de distribuție de probabilități, prin intermediul funcției Softmax.*
- *Funcția Softmax este importantă pentru că transformă rezultatele în numere ale căror sumă este 1, astfel este ușor de interpretat rezultatul clasificării.*



# Funcția de cost (engl. Loss Function )

- Am decis sa utilizăm *funcția de Entropie Încrucișată*, funcție provenita din Teoria Transmiterii Informației
- Aceasta este deosebit de utilă, deoarece *evaluează rapid și ușor rețeaua noastră, oferind valori mici pentru rezultatele apropiate de cele dorite.*
- **Metoda** de evaluare a rezultatului este **optimă pentru obiectivul de clasificare**, deoarece penalizează rezultatele proporțional mai mult cu cat se abat de la probabilitatea dorită.

$$- \frac{1}{N} \sum_{i=1}^N y_i \cdot \log(p(y_i)) + (1 - y_i) \cdot \log(1 - p(y_i))$$

Binary Cross-Entropy Loss:

$y_i$  – probabilitatea dorită de noi

$p(y_i)$  – probabilitatea calculată de model

Exemple de penalizări (1)

[0.05; 0.01; 0.06; 0.02; 0.03; 0.01; 0.01; 0.07; 0.72; 0.02]  
[0.00; 0.00; 0.00; 0.00; 0.00; 1.00; 0.00; 0.00; 0.00; 0.00]  
[ 0 ; 1 ; 2 ; 3 ; 4 ; 5 ; 6 ; 7 ; 8 ; 9 ]

Cifra 0 ->  $-(0 \cdot \ln(0.05) + (1 - 0) \cdot \ln(0.95)) = 0.051$

Cifra 1 ->  $-(0 \cdot \ln(0.01) + (1 - 0) \cdot \ln(0.99)) = 0.010$

Cifra 2 ->  $-(0 \cdot \ln(0.06) + (1 - 0) \cdot \ln(0.94)) = 0.062$

Cifra 3 ->  $-(0 \cdot \ln(0.02) + (1 - 0) \cdot \ln(0.98)) = 0.020$

Cifra 4 ->  $-(0 \cdot \ln(0.03) + (1 - 0) \cdot \ln(0.97)) = 0.030$

Cifra 5 ->  $-(1 \cdot \ln(0.01) + (1 - 1) \cdot \ln(0.99)) = 4.605$

Cifra 6 ->  $-(0 \cdot \ln(0.01) + (1 - 0) \cdot \ln(0.99)) = 0.010$

Cifra 7 ->  $-(0 \cdot \ln(0.07) + (1 - 0) \cdot \ln(0.93)) = 0.073$

Cifra 8 ->  $-(0 \cdot \ln(0.72) + (1 - 0) \cdot \ln(0.28)) = 1.273$

Cifra 9 ->  $-(0 \cdot \ln(0.02) + (1 - 0) \cdot \ln(0.98)) = 0.020$

$\Sigma$

$\rightarrow = 6.154 \cdot 1/10 = 0.6154$

*O valoare mare, care evidențiază  
faptul că modelul a greșit să clasifice  
corect cifra*

Exemple de penalizări (2)

[0.02; 0.005; 0.01; 0.005; 0.01; 0.92; 0.0225; 0.02; 0.03; 0.0025]  
[0.00; 0.00; 0.00; 0.00; 0.00; 1.00; 0.00; 0.00; 0.00; 0.00]  
[ 0 ; 1 ; 2 ; 3 ; 4 ; 5 ; 6 ; 7 ; 8 ; 9 ]

Cifra 0 ->  $-(0 \cdot \ln(0.02) + (1 - 0) \cdot \ln(0.98)) = 0.0202$

Cifra 1 ->  $-(0 \cdot \ln(0.005) + (1 - 0) \cdot \ln(0.995)) = 0.00501$

Cifra 2 ->  $-(0 \cdot \ln(0.01) + (1 - 0) \cdot \ln(0.99)) = 0.01005$

Cifra 3 ->  $-(0 \cdot \ln(0.005) + (1 - 0) \cdot \ln(0.995)) = 0.00501$

Cifra 4 ->  $-(0 \cdot \ln(0.01) + (1 - 0) \cdot \ln(0.99)) = 0.01005$

Cifra 5 ->  $-(1 \cdot \ln(0.92) + (1 - 1) \cdot \ln(0.08)) = 0.08338$

Cifra 6 ->  $-(0 \cdot \ln(0.0225) + (1 - 0) \cdot \ln(0.9775)) = 0.02277$

Cifra 7 ->  $-(0 \cdot \ln(0.02) + (1 - 0) \cdot \ln(0.98)) = 0.002$

Cifra 8 ->  $-(0 \cdot \ln(0.03) + (1 - 0) \cdot \ln(0.97)) = 0.003$

Cifra 9 ->  $-(0 \cdot \ln(0.0025) + (1 - 0) \cdot \ln(0.9975)) = 0.0025$

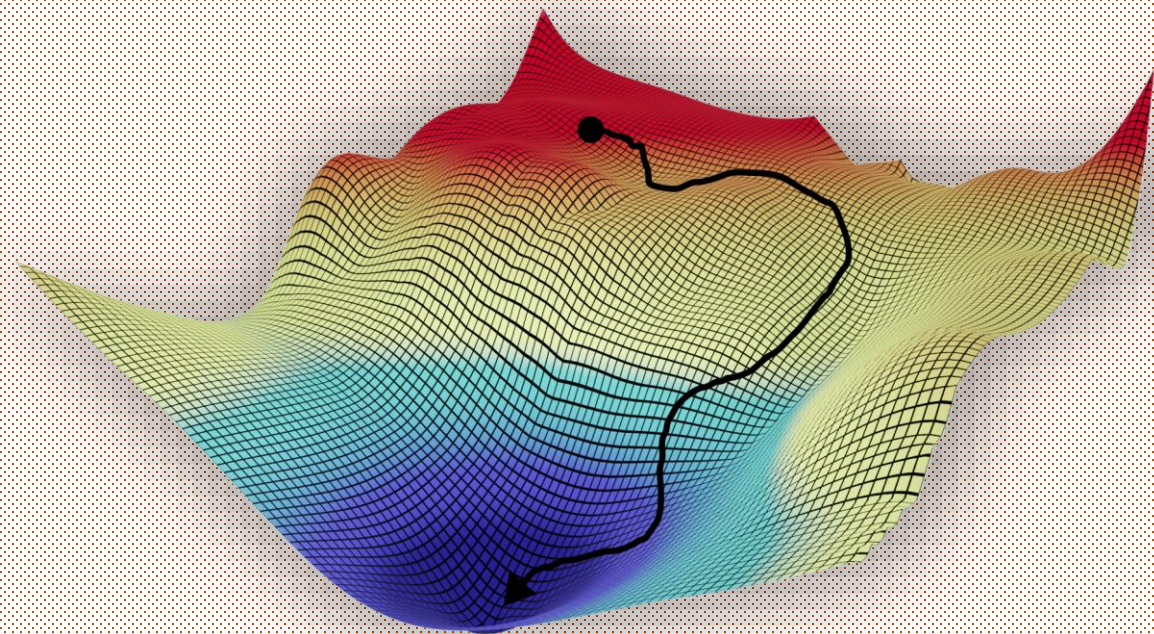
$\Sigma$

$\rightarrow = 0.16397 \cdot 1/10 = 0.016397$

*In cazul acesta, valoarea  
obținută e mult mai mică, ceea  
ce sugerează că evaluarea  
modelului este bună*

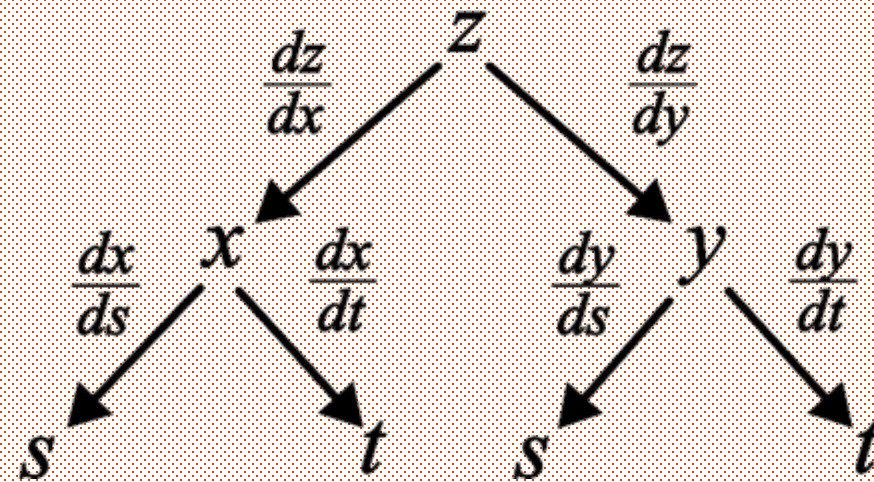
# Gradient Descent (Backpropagation)

- *Gradientul unei functii multivariabile reprezintă direcția cea mai abruptă cale spre un maxim local al funcției.*
- În cazul nostru, funcția evaluată este cea de cost, iar intenția noastră este de a o minimiza.
- Astfel, *vom scădea gradientul funcției, din argumentul acesteia (totalitatea ponderilor și valorilor de părtinire aferente rețelei), pentru a minimiza funcția de cost, și implicit îmbunătățirea rezultatelor*



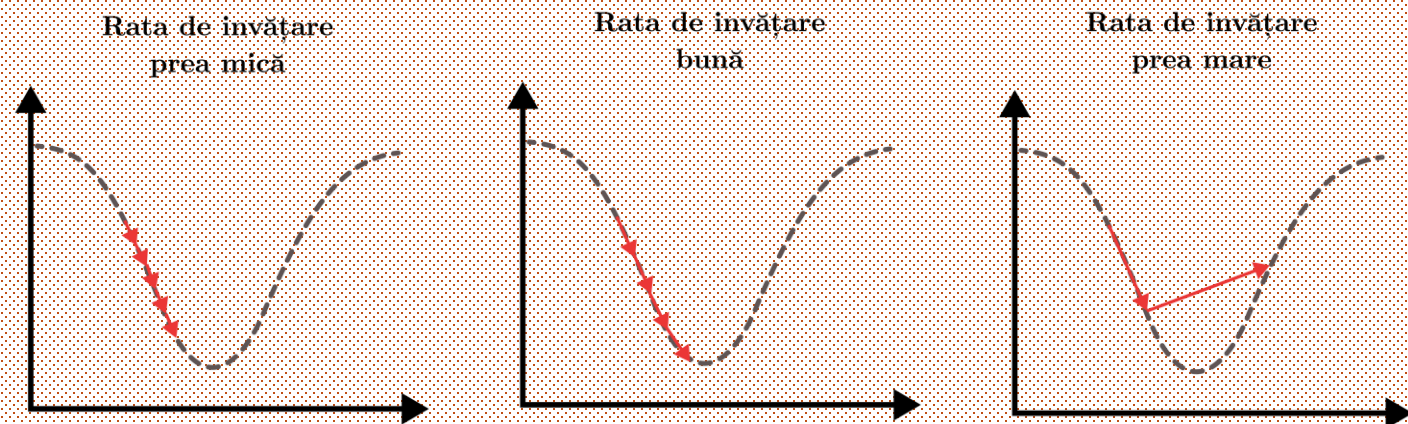
# Gradient Descent (Backpropagation)

- Pentru a îndeplini acest deziderat, *vom calcula derivatele parțiale ale funcției de cost, în raport cu fiecare variabilă.*
- Acest lucru, *este destul de laborios și intensiv din punct de vedere computațional*, deoarece evaluarea aportului fiecărei variabile, se rezumă la folosirea *Regulii de Derivare a funcțiilor compuse*, de mii sau milioane de ori, până ajungem la variabila în funcție de care am derivat



# Gradient Descent (Backpropagation)

- Gradientul, determină cea mai abruptă cale spre minimizarea funcției, în proximitatea argumentului nostru (*în cazul acesta, un vector  $n$  dimensional, unde  $n$  este un număr foarte mare*).
- Din acest motiv, vom desfășura modificarea valorii parametrilor „*din aproape în aproape*”, pentru a ne asigura că optăm mereu pentru cea mai bună cale de optimizare a funcției.
- Aici regăsim  $\alpha$  (*learning rate*), o valoare care tinde la 0, care are drept scop asigurarea îmbunătățirii optime a funcției de cost



*Alegerea unei rate de învățare optime este esențial învățării corecte*

$$W_{t+1} = W_t - \alpha \nabla C_{MEDIU}$$

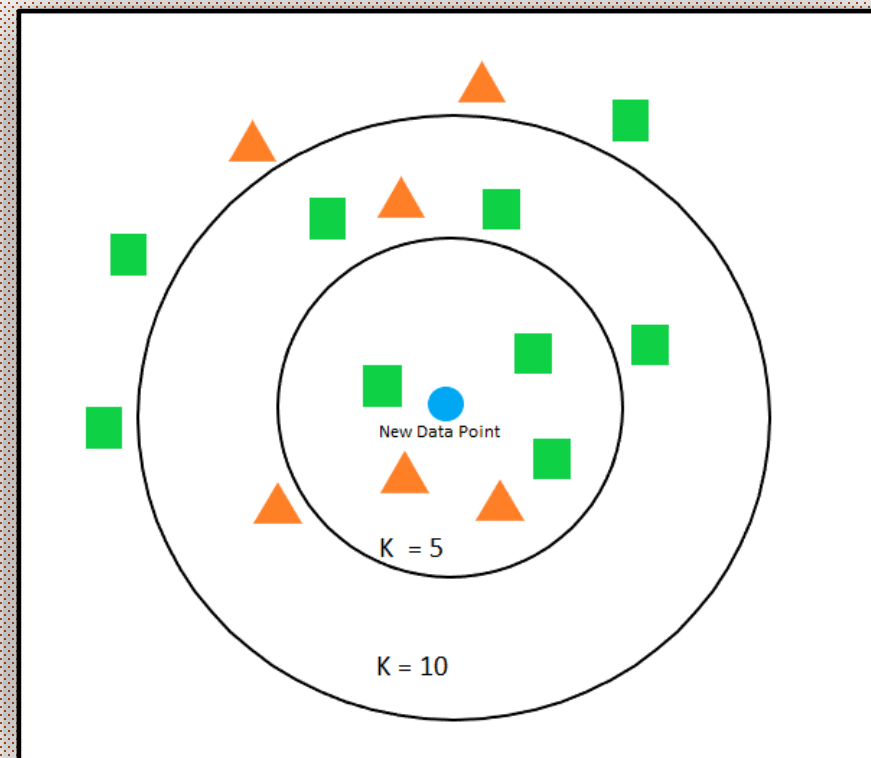
*Ajustarea parametrilor pe baza derivatelor parțiale*

# Caracterul Stocastic al antrenării Rețelelor Neurale

- *Algoritmul, presupune inițializarea tuturor ponderilor legăturilor și valorilor de părtinire aleatoriu.*
- *Caracteristica stocastică, provine din ideea că funcția de cost, va converge întotdeauna, indiferent de punctul de plecare, în aproximativ același interval de valori (foarte aproape de 0)*

# KNN (K-Nearest Neighbours)

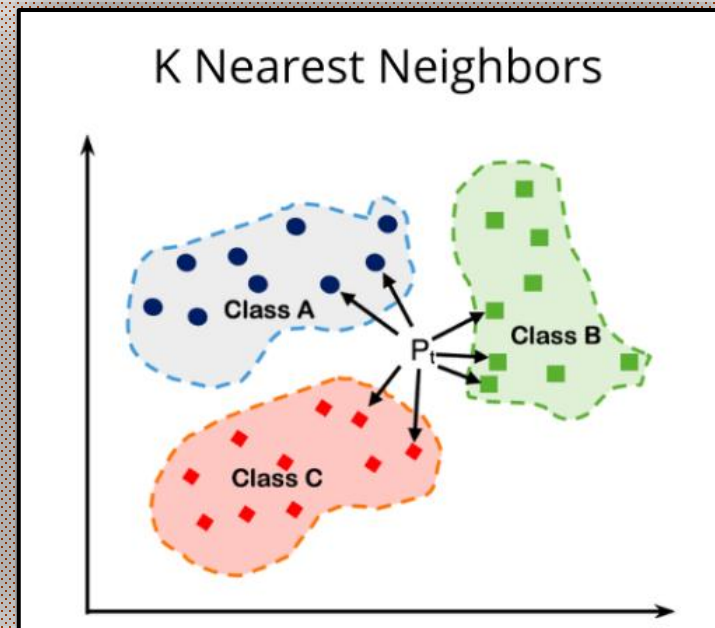
- *Algorithm determinist, provenit din domeniul Statisticii*, capabil să clasifice noi intrări, prin intermediul unui set de date mărimii relativ mică, în raport cu NN
- *K-ul se referă la numărul de vecini* necesari pentru a putea determina clasa/categoria noii intrări.
- De obicei, *K-ul este ales drept un număr impar*, pentru a evita cazuri de egalitate a numărului de vecini.
- Un K mic, este susceptibil valorilor aberante, în timp ce unul prea mare, omogenizează datele după care clasificăm



# Distanța Euclidiană

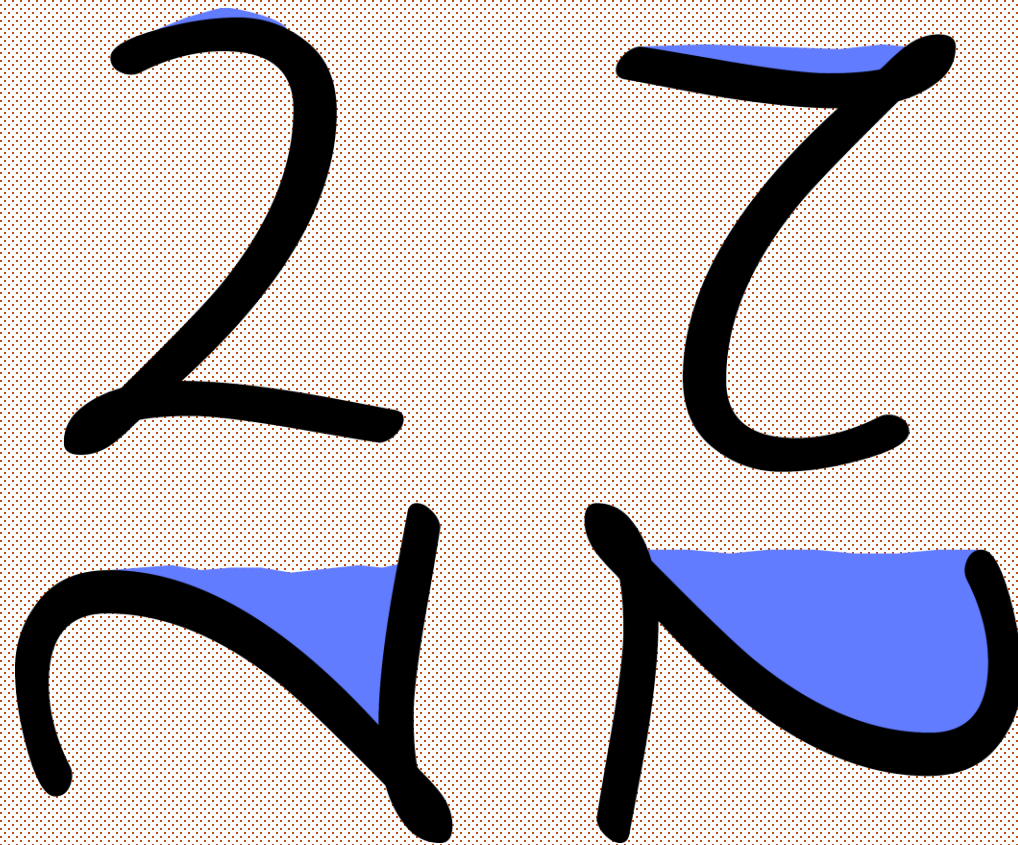
- Pentru a evalua cei mai apropiați vecini, *vom calcula distanțele vectorilor din setul nostru de date, raportate la vectorul de intrare.*
- Am ales această distanță, deoarece *este mai sensibilă valorilor continue, decât alte tipuri de distanțe* precum cea a Modulelor Diferențelor (Manhattan)

$$d(x, y) = \sqrt{\sum_{i=1}^n (y_i - x_i)^2}$$



# Modalitatea de Determinare a cifrelor

- Algoritmul nostru, *simulează turnarea unei găleți cu apă peste cifra noastră.*
- Acest lucru are loc, *din perspectiva fiecărei direcții cardinale și din toate direcțiile laolaltă (scufundarea numărului într-o găleată cu apă)*

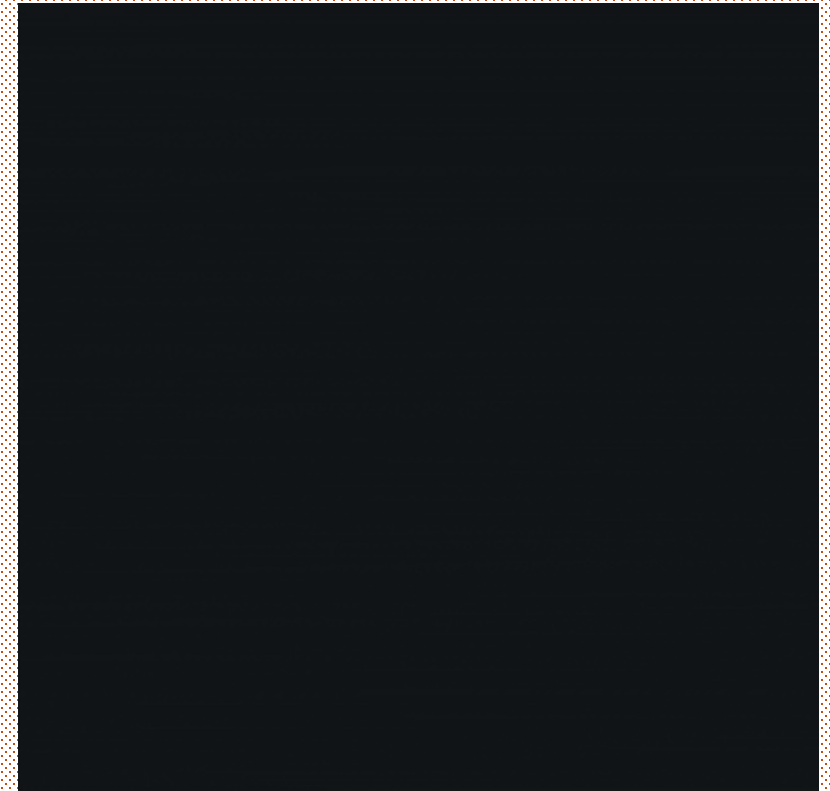


# Cum Determinăm Cantitatea de apă rămasă pe cifră?

- Pentru a simula turnarea unei găleți de apă peste cifră, vom folosi algoritmul

*Breadth-First-Search (Căutarea în lățime)*

- Vom interpreta întregul plan de 28x28 px ca fiind o matrice pătratică de 28x28
- *Vom inițializa algoritmul din direcția opusă celei din care este turnată apa*
- BFS se asigură că *doar pixelii de pe actualul nivel sau cel superior sunt marcați*
- *Pentru cifra 3, evaluarea cantității de apă rămasă, când apa cade din V, se rezumă la aplicarea BFS din E*



# Modalitatea de Determinare a cifrelor

- Pentru o mai bună evaluare și analiză, pentru fiecare direcție cardinală, *vom împărți numărul în două părți. Acest lucru, determină formarea unui vector de 9 dimensiuni*, fiecare element ținând cont de cantitatea de apă rămasă pe cifră, după următoarea formulă:

**Număr Pixeli Albaștri / (Număr Total de Pixeli / 2)**

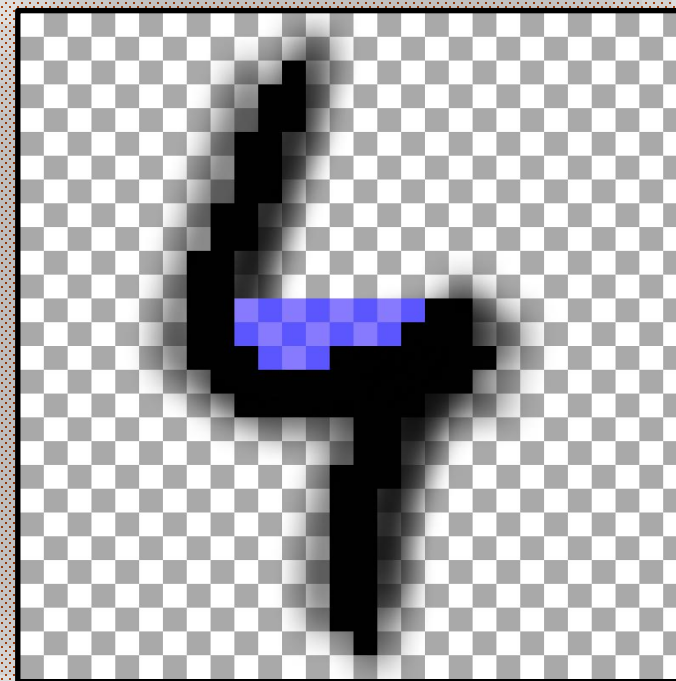
(Pentru direcțiile cardinale individuale)

**Număr Pixeli Albaștri / Număr Total de Pixeli**

(Pentru scufundarea numărului în apă)

- *Este un algoritm de tip “Lazy Learner”*

(setul de date utilizează aceeași procesare de date precum algoritmul, iar acesta le parcurge ulterior secvențial)



# Comparație

## KNN

- Acuratețe:  $\sim 80\%$
- Set de date:  $\sim 800$  de intrări
- Ușor de implementat atât dpdv. computațional cât și teoretic
- Este limitat dpdv. al utilizărilor
- Lazy Learner
- Determinist  
(etapa de antrenare)

## Rețea Neurală

- Acuratețe:  $95.47\%$
- Set de date:  $60000$  de intrări
- Difícil de implementat atât dpdv. computațional (necesită o perioadă lungă de antrenare), cât și teoretic (utilizează concepte din multe zone ale matematicii)
- Este universal dpdv. al utilizărilor
- Stochastic  
(etapa de antrenare)

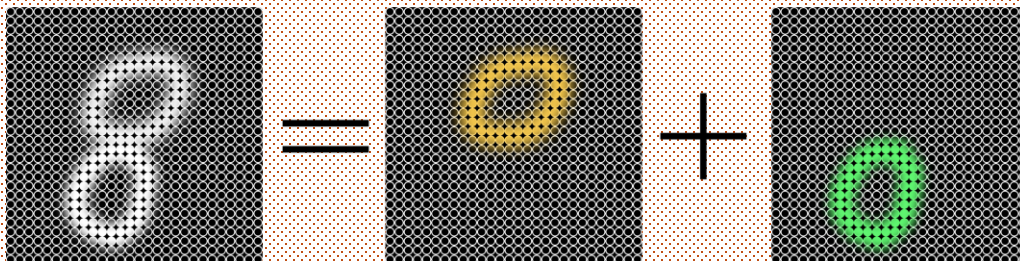
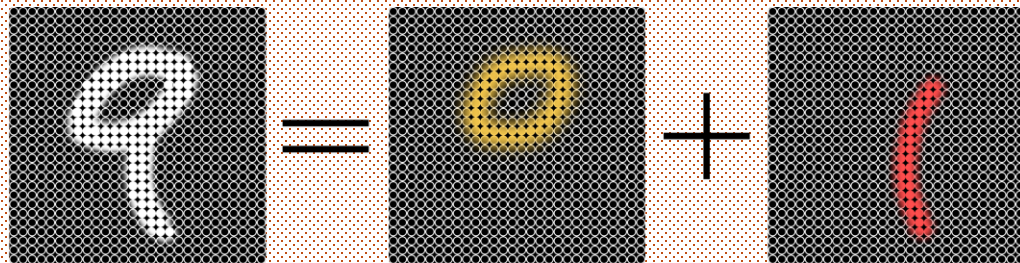
# Îmbunătățiri

## KNN

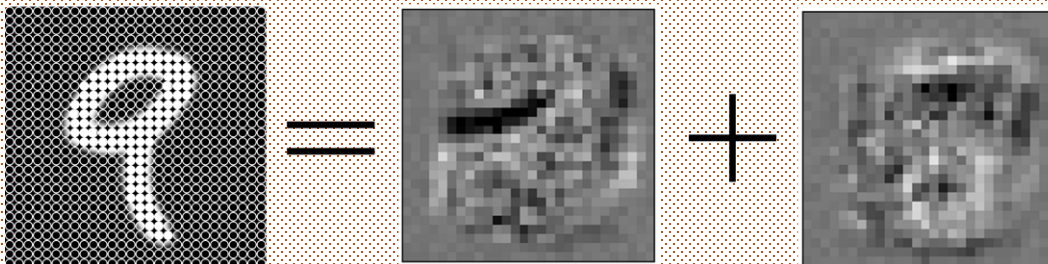
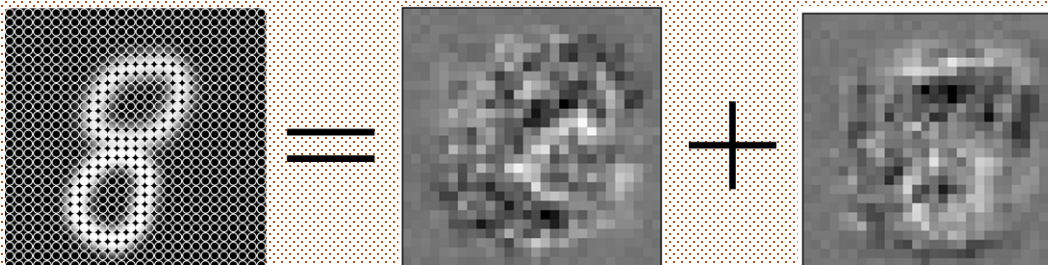
- Pentru a îmbunătăți rata de succes a algoritmului, *am putea adauga un alt tip de lichid*, care să interacționeze cu cifra după alte criterii/legi.
- *De exemplu*, am putea turna ulei peste acesta. *Uleiul este hidrofob*, ceea ce înseamnă că acesta *ar forma un strat deasupra apei, oferind mai multe date despre forma cifrei*
- O asemenea schimbare, *ar crește complexitatea algoritmului și mărimea datelor de procesat*, dar ar adăuga încă un strat folositor în clasificarea cifrei

## Rețea Neurală

- *Acestea sunt foarte versatile*, ceea ce ne permite să extindem scopul lor la detectarea mai multor obiecte într-o singură imagine/plan
- Prin *Rețelele Neurale Convolaționale*, putem să dezvoltăm un model mai complex, capabil să recunoască mai multe obiecte dintr-o singură imagine.
- *Transformerele*, sunt folosite în LLM-uri pentru a identifica contextul și relațiile dintre diferitele obiecte pe care le evaluăm
- *Rețelele Neurale Generative*, sunt capabile să reproducă și chiar să genereze noi obiecte (imagini, audio, text)



Exemple practice...



# Mulțumim!

*Autori: Bonciu Vlad-Matei, Beșliu Stefano*

*Academia de Studii Economice București, Facultatea de Cibernetică, Statistică și Informatică Economică*